CSI 5328 Artificial Intelligence
Homework 4 (Deep Neural Networks)

- **Submission Deadline**:
  - 1st submission: November 3, 2026, 11:59 pm CST
  - 2nd submission: November 17, 2026, 11:59pm CST (**for correction only**)
- **Submission Format**: The submitted file must include
  - Student name;
  - Bear ID;
  - Course name and number;
  - Homework number.

  The title of the submitted file must include
  - Homework number;
  - Student name.

  An example of titles of submissions would be:
  - `HW#_Name.pdf` (for write-up assignments);
  - `HW#_Name.zip` (for coding assignments).
- Please submit your homeworks through **Canvas**. Homeworks sent to the instructor's email will **NOT** be accepted.
- Late Submission: -10 penalty for the first 24-hour; -20 penalty for $\geq$ 24 hour; Zero score for $\geq$ 3 days.
- If you have any questions, please send them to the instructor, Dr. Chen Zhao, at chen_zhao@baylor.edu.
- If you have any questions on your **grades**, please send them to the TA, Xiaohui Chen, at xiaohui_chen1@baylor.edu.

Your submission needs to include:

1. Your Python scripts for each question (*i.e.,* All files end with .py).
2. Screenshots of outputs in a **single** *pdf* file.

For all questions, you must use the `PyTorch` library. Using other deep learning libraries (such as `TensorFlow`, `Keras`, *etc.*) will result in a grade of zero.

Question 1 [15pts]. Use `PyTorch` to train a regression model for the Boston dataset:

- Boston dataset: https://lib.stat.cmu.edu/datasets/boston

In the Boston dataset, "MEDV" is the dependent variable (or target variable), and the rest are independent variables. Split the dataset into a training set (70% samples) and a testing set (30% samples). Construct a simple neural network with two hidden layers:

- The first hidden layer must contain 16 units using the `ReLU` activation function
- The second layer must contain 32 units using the `tanh` activation function

Train the neural network using the split training dataset and report the mean squared errors (MSE) on the testing set. Then, create a single figure that includes the test loss curve over epochs. Finally, summarize your findings.

Question 2 [15pts]. Use `PyTorch` to train a classification model for the Titanic dataset:

- Titanic dataset: https://www.kaggle.com/c/titanic/data?select=train.csv

In the dataset, you need to focus on the "train.csv" file and take the "train.csv" as the Titanic dataset. "survived" is the dependent variable (or target variable), and the rest are independent variables. Split the dataset into a training set (80% samples) and a testing set (20% samples). Construct a simple neural network with two hidden layers:

- The first hidden layer must contain 5 units using the `ReLU` activation function
- The second layer must contain 3 units using the `tanh` activation function
- Recall that for a binary classification task, the output layer is activated by the `Sigmoid` function

Train the neural network using the split training dataset and report the classification accuracy, precision, recall, and F1 score of the testing sets.

Question 3 [20pts]. Given the input image in **Figure 1**, design two 3×3 filters/kernels to detect horizontal and vertical edges. For convenience, first convert the image to grayscale using `OpenCV`. Build a model with exactly one convolutional layer, one pooling layer, and one fully connected layer. For each kernel, visualize and report:



Figure 1: Question 3 image.

- The convolutional layer output (feature map);
- The `ReLU`-activated convolution output; and
- The pooled output.

In this question, you need to submit:

1. Your Python programming code; and
2. Six outputs images (two kernels by three visualizations) in total.

Question 4 [50pts]. Build a CNN-based image classification model in `PyTorch`. Use the `Fashion-MNIST` dataset, which can be found here: https://github.com/zalandoresearch/fashion-mnist. It consists of 60,000 training examples and 10,000 test examples. Each example is a 28×28 grayscale image labeled with one of 10 classes. When building your classification model, I recommend breaking the work into the following subtasks:

- Load the data set.
  - Please explore a few samples and visualize these images.
- Create a validation set and preprocess the images.
  - Split the training data set into a training set (90% samples) and a validation set (10% samples).
  - Convert images and labels into Torch format for both training and validation data sets.

- Implement CNNs using `PyTorch`.
  - Define the architecture with just 2 convolutional layers to extract features from the images and then use a fully connected dense layer to classify those features into their respective categories (i.e., two `Conv2d` layers and a linear layer).
  - Train the model for 25 epochs and show the validation losses by printing in the console. You are expected to see that the validation loss is decreasing as the epoch increases.
  - Visualize the training and validation losses by plotting them.
  - Show the accuracy of the model on the training and validation set.
- Generate predictions for the test set.
  - Load the test images.
  - Do the pre-processing steps on these images similar to what you did for the training images.
  - Generate predictions for the test set.

In this question, you need to submit:

1. Your Python programming code;
2. Five image visualization when loading the data set;
3. Plotting visualization of the training and validation losses;
4. Report the accuracy of your model on the training and validation set; and
5. Your predictions for the test images.